

Cấu trúc dữ liệu và thuật toán

Assignment 3

Giảng viên: Nguyễn Thị Tâm

§ Ngăn xếp - Hàng đợi §

Phần 1: Mục tiêu

- Nắm được định nghĩa, các thao tác, và độ phức tạp của các thao tác ứng với cấu trúc dữ liệu ngăn xếp (stack) và hàng đợi (queue)
- Nắm được cách triển khai cấu trúc dữ liệu ngăn xếp, hàng đợi trên mảng và danh sách liên kết.
- Hiểu các thuật toán sử dụng ngăn xếp để giải các bài toán như kiểm tra xâu ngoặc hợp lệ, biến đổi biểu thức trung tố thành tiền tố/hậu tố, tính toán biểu thức tiền tố/hậu tố bằng ngăn xếp.
- Tìm hiểu/sử dụng cấu trúc dữ liệu ngăn xếp/hàng đợi trong ngôn ngữ C++ (tùy chọn)

Phần 2: Lý thuyết

- Lý thuyết/Bài tập/Code mẫu cho Ngăn xếp: <https://www.geeksforgeeks.org/stack-data-structure/>
- Lý thuyết/Bài tập/Code mẫu cho Hàng đợi: <https://www.geeksforgeeks.org/queue-data-structure/>

Phần 3: Bài tập lý thuyết

- Chuyển các biểu thức trung tố sau sang biểu thức hậu tố ($\uparrow$ là kí hiệu của hàm mũ)
 - $A \uparrow B * C - D + E / F / (G + H)$
 - $((A + B) * C - (D - E)) \uparrow (F + G)$
 - $A - B / (C * D \uparrow E)$
 - $(a + b \uparrow c \uparrow d) * (e + f / d)$
 - $3 - 6 * 7 + 2 / 4 * 5 - 8$
 - $(A - B) / ((D + E) * F)$
 - $((A + B) / D) \uparrow ((E - F) * G)$
- Tính giá trị các biểu thức hậu tố sau, in nội dung stack ở mỗi bước của thuật toán:
 - P1: 5, 3, +, 2, *, 6, 9, 7, -, /, -
 - P2: 3, 5, +, 6, 4, -, *, 4, 1, -, 2, $\uparrow$, +
 - P3: 3, 1, +, 2, $\uparrow$, 7, 4, -, 2, *, +, 5, -
- Biến đổi biểu thức trung tố sau sang biểu thức tiền tố:
 - Đảo ngược chuỗi biểu thức trung tố

- Đảo dấu ‘(’ và dấu ‘)’
- Chuyển biểu thức đã đảo ngược sang biểu thức hậu tố
- Đảo ngược biểu thức hậu tố thành tiền tố

- $((A + 2) * (B + 4)) - 1$
- $Z - (((X + 1) * 2) - 5) / Y$
- $((C * 2) + 1) / (A + B)$
- $((A + B) * C - (D - E)) \uparrow (F + G)$
- $A - B / (C * D \uparrow E)$

4. Tính giá trị của các biểu thức tiền tố sau, in nội dung stack sau mỗi bước của thuật toán: **Làm tương tự như tính giá trị của biểu thức hậu tố, nhưng theo chiều ngược lại. (Lưu ý, thứ tự thực hiện phép toán với 2 toán tử ở đỉnh stack cũng phải tính theo chiều ngược lại.)**

- $+ * 2 + / 14 2 5 1$
- $- * 6 3 - 4 1$
- $+ + 2 6 + - 13 2 4$

5. Giả sử các ký tự ‘D’, ‘C’, ‘B’, ‘A’ được chèn vào queue theo đúng thứ tự đó và sau đó được loại bỏ từng ký tự, dãy nào ứng với các ký tự bị loại bỏ theo thứ tự thời gian?

- ABCD
- ABDC
- DCAB
- DCBA

6. Giả sử chúng ta đang triển khai queue vòng và có 10 phần tử đã đang được lưu trong queue ở các ô từ data[2] tới data[11]. Kích thước của mảng data là 42 phần tử. Nếu bổ sung một phần tử mới, phần tử đó sẽ được chèn ở đâu:

- data[1]
- data[2]
- data[11]
- data[12]

7. Giả sử S và Q là một stack và queue. Hãy vẽ nội dung của S và Q sau mỗi thao tác:

```

1  pushStack (S, 3)
2  pushStack (S, 12)
3  enqueue (Q, 5)
4  enqueue (Q, 8)
5  popStack (S, x)
6  pushStack (S, 2)
7  enqueue (Q, x)
8  dequeue (Q, y)
9  pushStack (S, x)
10 pushStack (S, y)
```

8. Vẽ nội dung của queue Q1 và Q2 cũng như stack S

```

1  S = createStack
2  Q1 = createQueue
3  Q2 = createQueue
4  enqueue (Q1, 5)
5  enqueue (Q1, 6)
6  enqueue (Q1, 9)
7  enqueue (Q1, 0)
8  enqueue (Q1, 7)
9  enqueue (Q1, 5)
```

```

10 enqueue (Q1, 0)
11 enqueue (Q1, 2)
12 enqueue (Q1, 6)
13 loop (not emptyQueue (Q1))
14     dequeue (Q1, x)
15     if (x == 0)
16         z = 0
17         loop (not emptyStack (S))
18             popStack (S, y)
19             z = z + y
20         end loop
21         enqueue (Q2, z)
22     else
23         pushStack (S, x)
24     end if
25 end loop

```

9. Nội dung queue Q là gì sau khi đoạn code sau được thực hiện với bộ dữ liệu 5, 7, 12, 4, 0, 4, 6, 8, 67, 34, 23, 5, 0, 44, 33, 22, 6, 0:

```

1 Q = createQueue
2 loop (not end of file)
3     read number
4     if (number not 0)
5         enqueue (Q, number)
6     else
7         x = queuerear (Q)
8         enqueue (Q, x)
9     end if
10 end loop

```

10. Nội dung queue Q1 và Q2 là gì sau khi đoạn code sau được thực hiện với bộ dữ liệu 5, 7, 12, 4, 0, 4, 6:

```

1 Q1 = createQueue
2 Q2 = createQueue
3 loop (not end of file)
4     read number
5     enqueue (Q1, number)
6     enqueue (Q2, number)
7     loop (not empty Q1)
8         dequeue (Q1, x)
9         enqueue (Q2, x)
10    end loop
11 end loop

```

11. Nội dung queue Q1 là gì sau khi đoạn code sau được thực hiện với bộ dữ liệu 5, 7, 12, 4, 0, 4, 6, 8, 67, 34, 23, 5, 0, 44, 33, 22, 6, 0:

```

1 Q1 = createQueue
2 S1 = createStack
3 loop (not end of file)
4     read number
5     if (number not 0)
6         pushStack (S1, number)
7     else
8         popStack (S1, x)
9         popStack (S1, x)
10        loop (not empty S1)
11            popStack (S1, x)
12            enqueue (Q1, x)

```

```

13         end loop
14     end if
15 end loop

```

12. Cho nội dung của queue Q1 và Q2, nội dung của queue Q3 là gì sau khi đoạn code sau được thực hiện:

Q1: 42 30 41 31 19 20 25 14 10 11 12 15

Q2: 4 5 4 10 13

```

1  Q3 = createQueue
2  count = 0
3  loop (not empty Q1 and not empty Q2)
4      count = count + 1
5      dequeue (Q1, x)
6      dequeue (Q2, y)
7      if (y equal count)
8          enqueue (Q3, x)
9      end if
10 end loop

```

Phần 4: Phần 4 bài tập thực hành

(1) Stack - Ngăn xếp

1. Hoàn thiện chương trình cài đặt Stack sử dụng LinkedList

```

1  // Giao diện StackInterface
2  public interface StackInterface<T> extends Iterable<T> {
3
4      public void push(T element);
5
6      public T pop();
7
8      public boolean isEmpty();
9  }
10
11  // Cài đặt stack dùng LinkedList
12  import java.util.Iterator;
13
14  public class LinkedListStack<T> implements StackInterface<T> {
15
16      class Node {
17          T element;
18          Node next;
19      }
20
21      Node stack = null;
22
23      @Override
24      public void push(T element) {
25          // TODO Auto-generated method stub
26      }
27
28
29      @Override
30      public T pop() {
31          // TODO Auto-generated method stub
32
33          return null;

```

```
34 }
35
36 @Override
37 public boolean isEmpty() {
38     // TODO Auto-generated method stub
39     return false;
40 }
41
42 @Override
43 public Iterator<T> iterator() {
44     // TODO Auto-generated method stub
45     return new StackIterator();
46 }
47
48 class StackIterator implements Iterator<T> {
49
50     private Node currentNode = stack;
51
52     @Override
53     public boolean hasNext() {
54         // TODO Auto-generated method stub
55         return currentNode != null;
56     }
57
58     @Override
59     public T next() {
60         // TODO Auto-generated method stub
61         T data = currentNode.element;
62         currentNode = currentNode.next;
63         return data;
64     }
65 }
66
67 }
```

2. Viết chương trình tính giá trị biểu thức có đầy đủ dấu ngoặc

```
1
2 public class FormulaEval {
3     public double eval(String formula)
4     {
5         return 0;
6     }
7 }
```

3. Kiểm tra tính hợp lệ của dấu ngoặc

```
1 public class Expression{
2     public boolean isValidExpr(String expr)
3     {
4         return false;
5     }
6     public static void main(String[] args)
7     {
8
9         Expression expr = new Expression();
10        String f = "a+b-c(3+a)";
11        System.out.println(expr.isValidExpr(f));
12
13    }
14 }
```

4. Đảo ngược stack - <https://practice.geeksforgeeks.org/problems/reverse-a-stack/1>
5. Xóa phần tử giữa của stack - <https://practice.geeksforgeeks.org/problems/delete-middle-element-of-a-stack/1>
 - Stack chỉ hỗ trợ hai thao tác chèn và xóa phần tử ở đỉnh stack.
 - Đề bài đã cho kích thước của stack, thực hiện các thao tác lấy ra các phần tử ở đỉnh stack cho đến khi phần tử ở giữa được lấy ra.
 - Sau đó đưa các phần tử đã được lấy ra trở lại stack trừ phần tử ở giữa.
6. Tính độ lồng nhau lớn nhất của các cặp ngoặc - <https://leetcode.com/problems/maximum-nesting-depth-of-the-parentheses/>
 - Chỉ quan tâm đến ngoặc mở và đóng
 - Sử dụng stack và đưa vào/lấy ra các ngoặc mở như bài kiểm tra xâu ngoặc hợp lệ.
 - Số phần tử lớn nhất trong stack chính là đáp án đề bài - hay nói một cách khác, đó chính là số ngoặc mở nhiều nhất có trong stack tại một thời điểm bất kỳ.
7. Biểu thức trung tố thành hậu tố - <https://practice.geeksforgeeks.org/problems/infix-to-postfix-1587115620/1>
 - Ref - <https://www.geeksforgeeks.org/stack-set-2-infix-to-postfix/>
8. Tính toán biểu thức hậu tố - <https://practice.geeksforgeeks.org/problems/evaluation-of-postfix-expression1735/1>
 - Ref - <https://www.geeksforgeeks.org/stack-set-4-evaluation-postfix-expression/>

Phần 5: Tự đánh giá

- ☐ Giải thích được các thao tác mà cấu trúc dữ liệu stack/queue hỗ trợ và độ phức tạp tương ứng.
- ☐ Triển khai được cấu trúc dữ liệu stack/queue bằng mảng/danh sách liên kết.
- ☐ Giải được các bài tập lý thuyết sử dụng các kiến thức cơ bản của stack/queue.
- ☐ Sử dụng được stack/queue để giải một số bài tập cơ bản